

# LLM *with* Retrieval-Augmented Generation

Tirtharaj Dash

Department of CS & IS  
BITS Pilani, Goa Campus, India  
[Homepage](#)

February 2026

# Outline

Recap: LLMs and their limitations

Retrieval-Augmented Generation (RAG)

The retrieval function

Generation with retrieved context

Evaluation of RAG systems

Summary: RAG interface

Demo: Jupyter

# LLMs: what we know so far

An autoregressive LLM factorises the joint distribution over a token sequence  $\mathbf{x} = (x_1, x_2, \dots, x_T)$  as:

$$P(\mathbf{x}; \theta) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}; \theta)$$

- ▶ The model parameters  $\theta$  **compress all knowledge** seen during training
- ▶ At inference: given a prompt  $q$ , the model generates

$$y^* = \arg \max_y P(y \mid q; \theta)$$

- ▶ Generation is purely *parametric*: no external lookup, no memory

Vaswani et al., *Attention is All You Need*, NeurIPS 2017

# Three fundamental limitations

## Stale knowledge

$\theta$  is fixed after training. Any fact that changes or emerges after the training cutoff is inaccessible to the model.

## Hallucination

$P(y \mid q; \theta)$  is a *probability distribution* over tokens, not a lookup table. The model generates fluent text even when it has no reliable basis for the claim.

## No attribution

There is no mechanism to trace *which* training document caused a particular output making verification impossible.

# A motivating example

## Query

*“What symbolic representation does Dash et al. use for molecules in their neurosymbolic drug discovery framework?”*

## LLM-only response (typical):

“Dash et al. likely represent molecules using SMILES strings fed into a graph neural network. . .”

- ▶ Sounds reasonable but may be entirely fabricated
- ▶ No source, no verifiability
- ▶ Worse: the paper may postdate the model's training cutoff

⇒ We need a way to supply the LLM with *relevant documents at query time*.

# The RAG idea

**Key insight:** instead of compressing all knowledge into  $\theta$ , maintain an *external knowledge base*  $\mathcal{K}$  and retrieve from it on demand.

## Standard LLM

$$y^* = \arg \max_y P(y \mid q; \theta)$$

Knowledge source:  $\theta$  only

## RAG

$$y^* = \arg \max_y P(y \mid q, \mathcal{D}_q; \theta)$$

Knowledge sources:  $\theta$  and  $\mathcal{D}_q$

where  $\mathcal{D}_q = \text{RETRIEVE}(q, \mathcal{K}, k)$  is a set of  $k$  document chunks retrieved from  $\mathcal{K}$ .

Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020

# Two phases of RAG

## Phase 1: indexing (offline)

Build a searchable index over the knowledge base  $\mathcal{K}$  once.

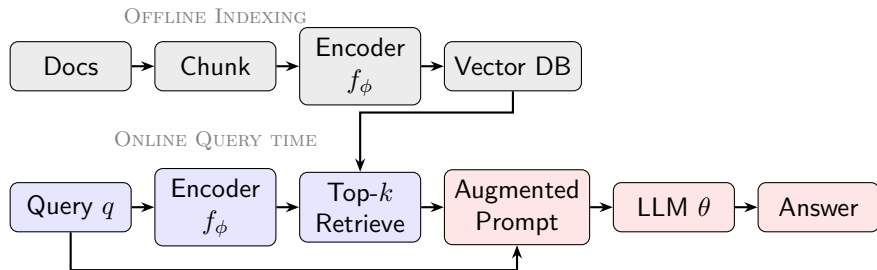
- (i) **Chunking**: split documents into manageable passages  $\{d_1, d_2, \dots, d_N\}$
- (ii) **Encoding**: map each chunk to a dense vector  $\mathbf{e}_i = f_\phi(d_i) \in \mathbb{R}^m$
- (iii) **Storing**: persist  $\{\mathbf{e}_i\}$  in a vector database

## Phase 2: query (online)

At inference time, for each user query  $q$ :

- (i) **Retrieve**: find top- $k$  chunks most similar to  $q$
- (ii) **Augment**: prepend retrieved chunks to the prompt
- (iii) **Generate**: pass augmented prompt to the LLM

# RAG pipeline: overview



## Augmented prompt:

$$\underbrace{[\text{System instruction}]}_{\text{role/task}} + \underbrace{[\mathcal{D}_q]}_{\text{retrieved chunks}} + \underbrace{[q]}_{\text{user query}}$$

## A toy retrieval example

Suppose the knowledge base contains three chunks:

1.  $d_1$ : *"Molecules are represented using molecular graphs."*
2.  $d_2$ : *"The model is trained using the Adam optimiser."*
3.  $d_3$ : *"Drug candidates are evaluated on benchmark datasets."*

**Query:**

*"How are molecules represented?"*

After embedding the query and chunks:

$$\text{sim}(q, d_1) = 0.91, \quad \text{sim}(q, d_2) = 0.18, \quad \text{sim}(q, d_3) = 0.42$$

Therefore, for  $k = 1$ :

$$\mathcal{D}_q = \{d_1\}$$

⇒ Retrieval converts a large knowledge base into a small, relevant context for the LLM.

## Dense retrieval: semantic embeddings

The encoder  $f_\phi$  maps text to a dense vector in  $\mathbb{R}^m$  such that *semantically similar* texts are geometrically close.

**Cosine similarity** between query  $q$  and chunk  $d_i$ :

$$\text{sim}(q, d_i) = \frac{f_\phi(q)^\top f_\phi(d_i)}{\|f_\phi(q)\| \|f_\phi(d_i)\|} \in [-1, 1]$$

**Retrieval:** return the  $k$  chunks with highest similarity

$$\mathcal{D}_q = \arg \max_{S \subseteq \mathcal{K}, |S|=k} \sum_{d_i \in S} \text{sim}(q, d_i)$$

In practice: **FAISS** (Facebook AI Similarity Search) enables approximate nearest-neighbour search at scale.

Johnson et al., IEEE TBDM 2019

## Sparse vs. dense vs. hybrid retrieval

| Property          | Sparse (BM25)  | Dense                 | Hybrid |
|-------------------|----------------|-----------------------|--------|
| Representation    | TF-IDF weights | $\mathbb{R}^m$ vector | Both   |
| Matching          | Exact term     | Semantic              | Both   |
| Paraphrase recall | Poor           | Good                  | Good   |
| Rare terms        | Good           | Can miss              | Good   |
| Compute           | Fast           | Moderate              | Higher |

**Reciprocal Rank Fusion (RRF)** for hybrid scoring:

$$\text{score}_{\text{RRF}}(d_i) = \frac{1}{r_{\text{sparse}}(d_i) + c} + \frac{1}{r_{\text{dense}}(d_i) + c}$$

where  $r(\cdot)$  is rank and  $c$  is a smoothing constant (typically 60).

# Chunking strategy matters

The granularity of  $\{d_1, \dots, d_N\}$  directly affects retrieval quality.

## **Fixed-size**

Split every  $w$  words with overlap  $\delta$ .

Simple but may cut sentences.

## **Sentence-based**

Chunk at sentence boundaries.

Semantically cleaner; variable length.

## **Recursive / hierarchical**

Respect document structure (sections, paragraphs).

Preserves context best.

## **Chunk size trade-off:**

- ▶ *Too small*: retrieved chunk lacks context  $\Rightarrow$  poor generation
- ▶ *Too large*: floods the LLM context window; retrieval precision drops

## Conditional generation

Given retrieved chunks  $\mathcal{D}_q = \{d_1^*, \dots, d_k^*\}$ , the LLM generates:

$$y^* = \arg \max_y P(y \mid q, d_1^*, \dots, d_k^*; \theta)$$

Using the chain rule, this factorises token-by-token as before:

$$P(y \mid q, \mathcal{D}_q; \theta) = \prod_{t=1}^{|y|} P(y_t \mid y_{<t}, q, \mathcal{D}_q; \theta)$$

**Nothing changes in the LLM itself.**

The retrieved chunks are simply prepended to the input;  $\theta$  remains fixed during inference.

⇒ RAG is a *prompting strategy*, not a fine-tuning strategy.

# The context window constraint

The LLM can only process a finite number of tokens at once (the *context window*).

## Budget decomposition:

$$L_{\text{total}} = L_{\text{system}} + \underbrace{\sum_{i=1}^k |d_i^*|}_{\text{retrieved context}} + L_q + L_y \leq L_{\text{max}}$$

Increasing  $k$  gives more grounding but:

- ▶ Consumes more of the context window
- ▶ Risks burying the relevant chunk among noise (*lost-in-the-middle* problem)
- ▶ Increases latency

**Practical choice:**  $k \in \{3, 5\}$  for most research QA tasks.

Liu et al., *Lost in the Middle: How LMs Use Long Contexts*, TACL 2024

# Reranking: improving retrieval precision

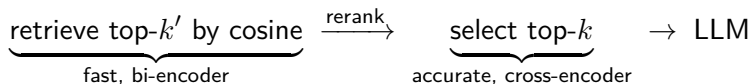
First-stage retrieval (cosine similarity) is fast but imprecise.

**Reranker:** a cross-encoder that scores each  $(q, d_i)$  pair jointly:

$$s_i = g_\psi(q, d_i) \in \mathbb{R}$$

- ▶ Sees *both* query and passage together  $\Rightarrow$  much more accurate
- ▶ Too slow to run over all  $N$  chunks; applied only to top- $k'$  from first stage

**Two-stage pipeline:**



Nogueira & Cho, *Passage Re-ranking with BERT*, arXiv 2019

# Where can a RAG system fail?

A RAG system has two major opportunities for error.

## 1. Retrieval failure

The relevant information exists in  $\mathcal{K}$ , but the retriever does not return it.

## 2. Generation failure

The correct information is retrieved, but the LLM ignores it, misinterprets it, or generates an unsupported answer.

⇒ Good retrieval does not automatically imply a correct answer.

This is why retrieval and generation should be evaluated separately.

## RAG vs. fine-tuning vs. long context

| Property                      | RAG     | Fine-tune | Long context |
|-------------------------------|---------|-----------|--------------|
| Updates knowledge             | Dynamic | Static    | Dynamic      |
| Trains $\theta$               | No      | Yes       | No           |
| Scales to large $\mathcal{K}$ | Yes     | N/A       | No           |
| Attribution                   | Yes     | No        | Partial      |
| Cost (inference)              | Low     | Low       | High         |
| Hallucination                 | Reduced | Moderate  | Moderate     |

RAG is the preferred approach when the knowledge base is *large*, *dynamic*, or requires *traceable sources*.

# How do we evaluate RAG?

Two sub-systems to evaluate independently and end-to-end:

## Retrieval quality:

- ▶ *Recall@k*: fraction of relevant chunks in top- $k$  retrieved
- ▶ *Mean Reciprocal Rank (MRR)*:

$$\text{MRR} = \frac{1}{|Q|} \sum_q \frac{1}{\text{rank of first relevant chunk}}$$

## Generation quality (RAGAS framework):

- ▶ *Faithfulness*: is the answer supported by  $\mathcal{D}_q$ ?
- ▶ *Answer relevancy*: does the answer address  $q$ ?
- ▶ *Context recall*: do retrieved chunks cover the ground-truth answer?

Es et al., *RAGAS: Automated Evaluation of RAG*, EACL 2024  
(arXiv:2309.15217)

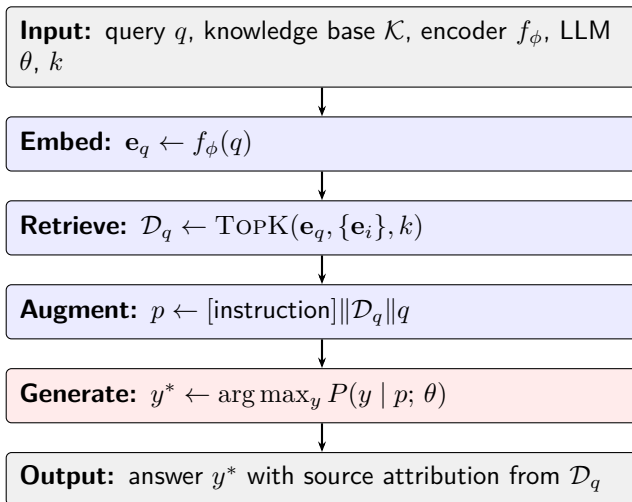
# The key idea to remember

## LLM + Search + Context

- ▶ The **retriever** decides what information the LLM sees
- ▶ The **LLM** decides how to use that information
- ▶ The external knowledge base can be updated without retraining the LLM

$q \longrightarrow \text{retrieve relevant evidence} \longrightarrow \text{LLM} \longrightarrow \text{answer}$

# RAG inference



**Knowledge base  $\mathcal{K}$ :** published papers on neurosymbolic AI and drug discovery.

### Queries of interest:

- ▶ *“What symbolic representation is used for molecules?”*
- ▶ *“Which datasets are used for drug-likeness prediction?”*
- ▶ *“How is background knowledge encoded in the ILP framework?”*
- ▶ *“What is the performance vs. GNN baselines?”*

**Comparison:** LLM-only answer vs. RAG-augmented answer with source chunk displayed.

We will observe: LLM hallucinates method details; RAG cites exact text from the correct paper.